- This lab will cover sorting algorithms and more recursion.
- You may want to refer to the text and your lecture notes during the lab as you solve the problems.
- When approaching the problems, think before you code. Doing so is good practice and can help you lay out possible solutions.
- Think of any possible test cases that can potentially cause your solution to fail!
- Your TAs are available to answer questions in lab, during office hours, and on Edstem.

Vitamins

---

1. For each of the following code snippets (15 minutes):

   A. Trace the execution of each code snippet and write down all outputs in order i.e. Write down what the print statements will output.
   B. Give an apt name for each of the functions.

(a)
```
def question1(n):
      if n == 0:
            return 1
      result = n * question1(n - 1)
print(n, result)
      return result

print(question1(4))
```

(b)
```
def question2(n):
      if n <= 1:
            return n
      print(n)
      return question2(n - 1) + question2(n - 2)
print("value", question2(4))
```

2. For each of the following code snippets:
   A. Trace the execution of each code snippet and write down output of the print statement.
   B. Draw the recursion tree that represents the execution process of the function, and the cost of each call.
   C. Fill in the tables with relevant values as prompted by the column names as you draw the recursion tree. For runtime determine the asymptotic runtime of each function for a general positive integer input n.

The level 0 row of the first table has been completed as an example.

(a)
```
def question3(n):
    if n <= 1:
        return 1
    return question3(n // 2) + question3(n // 2)

print("value", question3(23))
```

Level	# calls	Input size per call	Local work per call	Work on level
0	1	23	1	1
1				
2				
⋮	⋮	⋮	⋮	⋮
Total				
Runtime				

(b)
```
def question4(n):

    if n <= 1:
        return 1

    total = 0
    for i in range(n):
        total += i

    return total + question4(n // 2) + question4(n // 2)
print(question4(8))
```

Level	# calls	Input size per call	Local work per call	Work on level
0				
1				
2				
⋮	⋮	⋮	⋮	⋮
Total				
Runtime				

(c)
```
def question5(n):

    if n <= 1:
        return 1
    count = 0
    for i in range(n):
        for j in range(n):
            count += 1
    return count + question5(n // 2) + question5(n // 2)

print(question5(6))
```

Level	# calls	Input size per call	Local work per call	Work on level
0				
1				
2				
⋮	⋮	⋮	⋮	⋮
Total				

Coding

In this section, it is strongly recommended that you solve the problem on Write a function that takes a list of length n containing every integer from 0 to n−1 exactly once. Sort the list in ascending order in place. Do not use built-in sorting functions. The function must return None. to n-1 and **sorts them in place**. You can assume duplicate elements do not exist within the input list.Implementation runtime must be linear. (15 minutes)

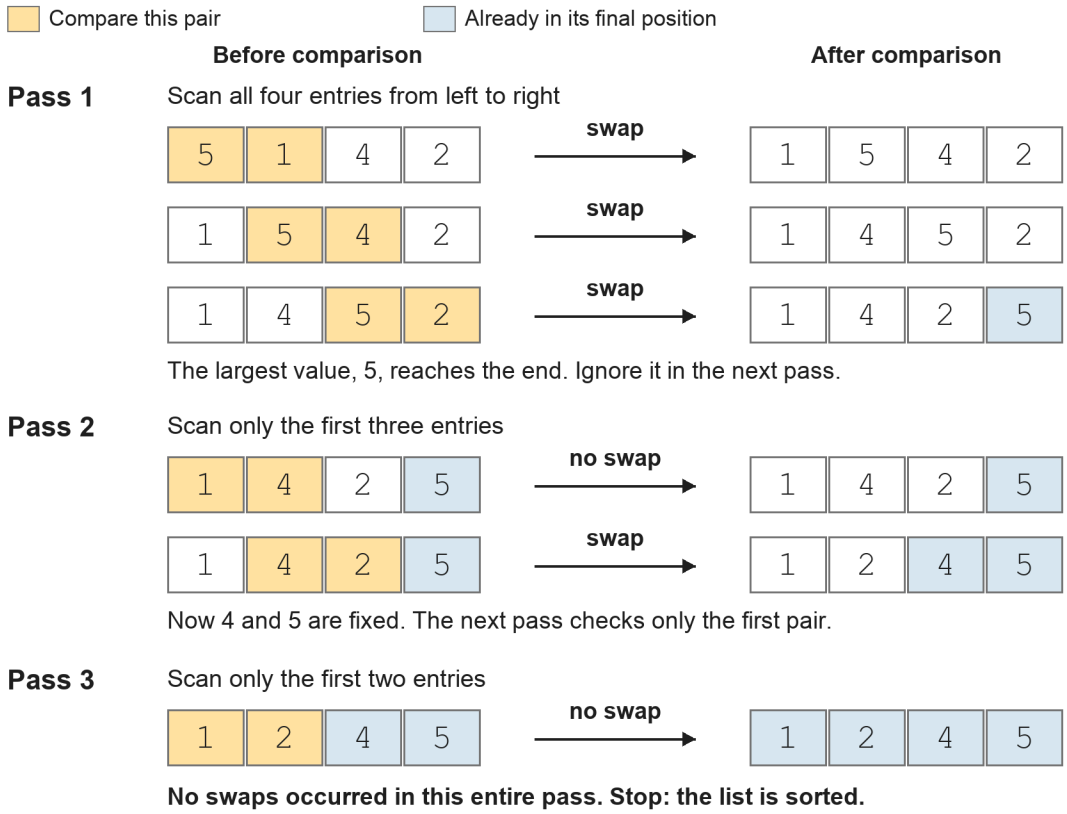
```
ex) lst = [0,2,1,3] , SortLst(lst) -> [0,1,2,3]

    def SortLst(lst):
      """
      : lst type: list[int]
      : return type: None
      ""
```

Bubble sort

Bubble sort repeatedly scans a list from left to right. Compare neighboring values and swap them only when the left value is greater. One complete scan of the unsorted portion is called a pass.

The diagram traces [5, 1, 4, 2]. Follow the yellow pair in each row; blue entries can be ignored in later passes.



Why the boundary moves left

Each pass places the largest remaining value in its final position.

The next pass therefore needs one fewer comparison.

A large value can move several positions to the right during one pass. A pass fixes the largest remaining value, but it may leave the earlier entries unsorted.

2. Write a function that sorts a list of integers in ascending order in place using bubble sort. Stop early if a full pass makes no swaps. Do not use sort or sorted. The function must return None. Worst-case implementation runtime must be O(n^2). (15 minutes)

```
ex) lst = [5, 1, 4, 2]
bubble_sort(lst) returns None and changes lst to [1, 2, 4, 5].

def bubble_sort(lst):
    """
    : lst type: list[int]
    : return type: None
    """
```

3. Write a recursive function that takes in a nonnegative integer n and returns True if it is a power of 2 and False otherwise. Do not use loops, logarithms, exponentiation, or strings. Treat 1 as a power of 2 and 0 as not a power of 2. Implementation runtime must be O(log n) for n >= 2. (10 minutes)

```
ex) isPowerOfTwo(16) returns True.
isPowerOfTwo(12) returns False.
      isPowerOfTwo(1) returns True; isPowerOfTwo(0) returns False.

def isPowerOfTwo(n):
    """
    : n type: nonnegative int
    : return type: bool
             """
```

4. Write a recursive function that mutates a list so that all even integers precede all odd integers within the inclusive range low through high. Relative order does not need to be preserved but values outside the range must remain unchanged.
Assume valid indices, or low > high for an empty range. Do not use loops, slicing, or sorting. Implementation runtime must be O(n), where n is the range length. (15 minutes)

```
ex) lst = [4, -5, 2, 3, -1, -6, 7, 9, 0]
split_parity(lst, 0, 8) may produce
      [4, 0, 2, -6, -1, 3, 7, 9, -5].

def split_parity(lst, low, high):
    """
    : lst type: list[int]
    : low, high type: int
```

```
    : return type: None
              """
```

5. A nested list contains integers and other nested lists. Write a recursive function that returns the sum of every integer in the nested list. Do not modify the input. You may use for loops and isinstance, but do not use sum or flatten the list. Implementation runtime must be linear in the total number of entries across all lists. (15 minutes)

```
ex) nested_sum([[1, 2], 3, [4, [5, 6]], []]) returns 21.
nested_sum([[], [-2, [0]], 7]) returns 5.

def nested_sum(lst):
    """
    : lst type: nested list of integers
    : return type: int
    """
```

6. A nested list contains integers and other nested lists. Write a recursive function that returns a new flat list of every integer from left to right. Do not modify the input. You may use for loops, isinstance, and a helper function. Use append to build the result; do not repeatedly concatenate lists or use a generator. Implementation runtime must be linear in the total number of entries across all lists. (20 minutes)

```
ex) flatten([[1, 2], 3, [4, [5, 6]], []]) returns [1, 2, 3, 4, 5, 6].
flatten([[], [-2, [0]], 7]) returns [-2, 0, 7].
      flatten([]) returns [].

def flatten(lst):
    """
    : lst type: nested list of integers
    : return type: list[int]
              """
```

7. Write the function balanced_paratheses that returns True if a string containing only '(' and ')' has balanced parentheses, and False otherwise. You have to write a recursive helper function. Every opening parenthesis must have a later closing partner, and no closing parenthesis may appear too early. The empty string is balanced. Do not use loops or slicing. Implementation runtime must be O(n), where n is the string length. (20 minutes)

```
ex) balanced_parentheses('(()())') returns True.

balanced_parentheses(')(') returns False.

balanced_parentheses('(()') returns False.

def balanced_parentheses(text):
    """
    : text type: str
    : return type: bool
    """
```

Optional

8. Write a recursive generator function that takes in a nested list and yields each integer from left to right. Do not construct a flattened list or modify the input. You may use for loops, isinstance, and yield from. Values should be produced as the generator is iterated. (20 minutes)

```
ex) list(yield_flattened([1, [2, [], [3]], 4])) returns [1, 2, 3, 4].
     list(yield_flattened([])) returns [].

def yield_flattened(lst):
    """
    : lst type: nested list of integers
    : yield type: int
             """
```